

CS-600: Software Design and Development

4-2 Short Paper: Analyzing Life Cycle Models for Sustainable Development

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

May 11, 2026

Software Development Life Cycle Recommendation for a Research Data Management System

Selecting the correct software development life cycle (SDLC) process is one of the most important decisions in software engineering because it affects how efficiently a system is developed, kept, and improved over time (Sommerville, 2016). For this project, the customer requires a secure software application that combines a visual user interface with a large back-end database capable of storing several hundred million research records. The system must support create, read, update, and delete (CRUD) functionality, advanced search and filtering using regular expressions, and scheduled report generation. Because the project holds complex requirements, large-scale data storage, and long-term maintenance needs, the most proper software development life cycle model for this application is the Agile SDLC process combined with DevOps practices. This approach supports flexibility, continuous improvement, ongoing testing, and efficient long-term maintenance (Schwaber, 2004). Agile allows developers to work in small iterations while DevOps supports automation, deployment, monitoring, and maintenance after release (Kim et al., 2021).

Key Features of the Agile Life Cycle Process

One of the key features of Agile is its iterative development. Instead of building the entire application at once, the project is divided into smaller phases called sprints (Schwaber, 2004). Each sprint focuses on a specific set of features and produces a working part of the application. This is beneficial for the research system because stakeholders can review progress regularly and

request adjustments as development continues. Another important feature is continuous stakeholder involvement. Researchers and project stakeholders can provide feedback throughout development instead of waiting until the end of the project. Since the application handles observational findings, sensor readings, search capabilities, and reporting tools, regular feedback helps ensure the system meets user expectations and scientific requirements. Agile also supports continuous testing and integration. Because the system stores sensitive research data and must handle hundreds of millions of records, testing performance, security, and database functionality throughout development is essential (Sommerville, 2016). Automated testing can show problems early before they become larger issues later in the project. DevOps practices further improve the life cycle process by supporting automated deployments, monitoring, and maintenance. Continuous Integration and Continuous Deployment (CI/CD) pipelines help developers release updates safely and efficiently (Kim et al., 2021). Monitoring tools can track system performance, database health, and security vulnerabilities after deployment.

Supporting Short-Term Development and Long-Term Maintenance

The Agile and DevOps approach supports both short-term development goals and long-term maintenance requirements. In the short term, Agile allows the team to quickly develop the core functionality of the application. During early sprints, developers can focus on creating database architecture, CRUD operations, user authentication, and secure visual interfaces. Later sprints can add advanced regex filtering and report scheduling features. This phased approach reduces project risk because features are developed and tested gradually (Schwaber, 2004). Agile also improves communication between developers, database administrators, testers, and stakeholders.

Frequent sprint reviews help name misunderstandings early and allow the team to adapt to changing requirements.

For long-term maintenance, DevOps provides continuous monitoring, automated testing, and streamlined updates (Kim et al., 2021). Since the system will likely evolve over many years, maintenance is critical. New scientific requirements, database growth, and updated security standards will require changes. Agile makes it easier to add new features incrementally without redesigning the entire system. Furthermore, automated monitoring tools can find performance issues as the database grows. Regular updates and patches can be deployed with minimal downtime. This is especially important because the application manages large-scale research data that users may need to access continuously.

Managing Software Defects Across the Application Life Cycle

The Agile process is highly effective for managing software defects because testing occurs continuously throughout development instead of only at the end (Sommerville, 2016).

Developers can show bugs during each sprint and correct them at once. This reduces the risk of major system failures later in the project. Automated testing tools further improve defect management. Unit tests confirm individual functions; integration tests verify communication between components, and performance tests ensure the database can handle hundreds of millions of records efficiently. DevOps monitoring tools also help detect problems after deployment. If performance slows down or security vulnerabilities appear, the development team can quickly respond with updates or patches. Continuous monitoring reduces downtime and improves system reliability. Agile retrospectives encourage teams to review earlier sprint challenges and improve

development processes over time (Schwaber, 2004). This continuous improvement mindset helps reduce repeated defects and strengthens overall software quality.

Managing New Feature Requests

One of the greatest advantages of Agile is its flexibility in handling new feature requests. Scientific research projects often evolve, and stakeholders may request more reporting tools, advanced analytics, or new search capabilities in the future. Because Agile development occurs in iterative sprints, new features can be added to the project's backlog and prioritized for future releases. Developers do not need to redesign the entire system to implement updates. Instead, they can gradually expand functionality while keeping system stability. DevOps practices also simplify feature implementation by automating testing and deployment. New features can be tested in staging environments before being safely released to production (Kim et al., 2021). This reduces the risk of introducing errors into the live system.

Comparison to the Waterfall Life Cycle Model

An alternative life cycle model for this project is the Waterfall model. Waterfall follows a linear sequence where each phase must be completed before moving to the next stage. While this model works well for projects with fixed and predictable requirements, it is less suitable for this research application. One major weakness of Waterfall is limited flexibility. If stakeholders request changes after development begins, implementing them can be difficult and expensive. In contrast, Agile allows continuous feedback and adaptation throughout the project (Schwaber, 2004). Waterfall also delays testing until later stages of development. This increases the risk of

discovering major defects late in the project. Agile's continuous testing process reduces this risk significantly.

However, Waterfall does provide advantages in documentation and structure. Some organizations prefer Waterfall because timelines and requirements are clearly defined from the beginning. Despite this, the large-scale database requirements, evolving user needs, and long-term maintenance demands make Agile the more proper choice for this system.

Benefits and Challenges of Agile in a Professional Setting

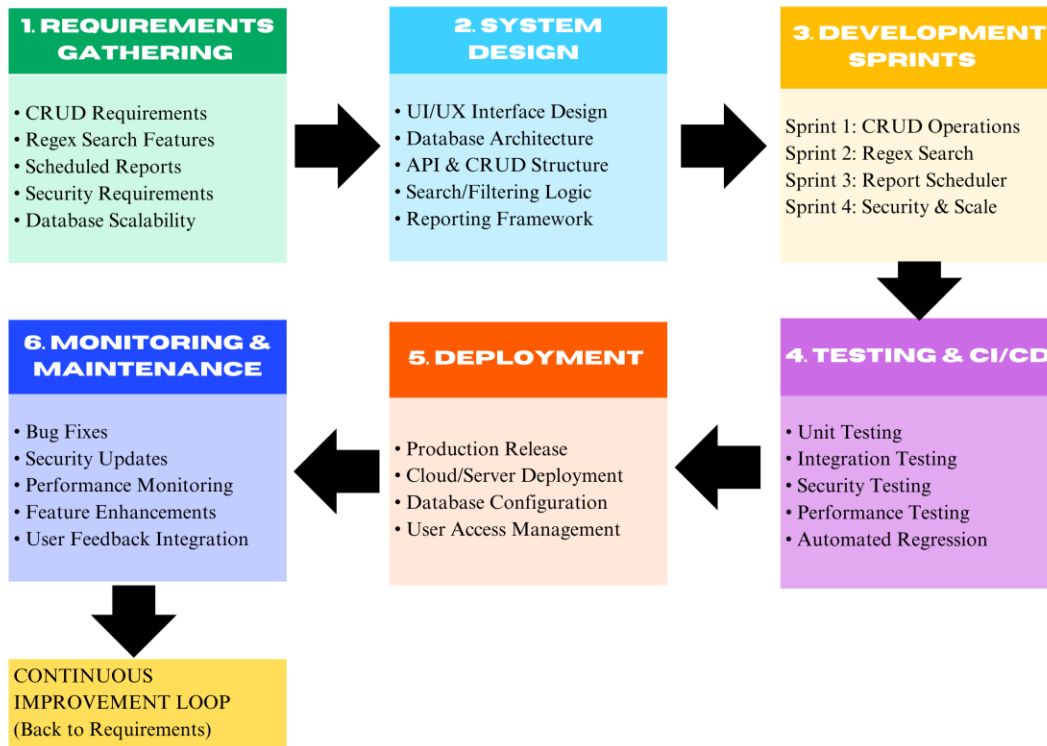
Implementing Agile and DevOps provides many professional benefits. One major advantage is improved adaptability. Teams can respond quickly to changing requirements, scientific discoveries, or stakeholder feedback. This flexibility is essential for research-based systems that may evolve. Another benefit is the faster delivery of working software. Instead of waiting until the entire project is complete, stakeholders receive usable system components during each sprint. This improves collaboration and increases user satisfaction. Continuous testing and deployment also improve software quality and security (Kim et al., 2021). Problems can be detected and corrected earlier, reducing overall project risk. Despite these advantages, Agile also presents challenges. Frequent communication and collaboration require strong teamwork and organization. Without effective planning, projects can experience a scope of creep where requirements continue to expand. Another challenge is supporting documentation. Because Agile focuses heavily on rapid development, teams must ensure documentation stays updated throughout the process. Large organizations may also require other coordination between multiple development teams. Even with these challenges, the benefits of flexibility, continuous

improvement, and long-term maintainability make Agile and DevOps the best choice for this project.

To conclude, the Agile SDLC process combined with DevOps practices is the right life cycle model for this research data management system. The approach supports iterative development, continuous testing, stakeholder collaboration, and long-term maintenance (Schwaber, 2004; Kim et al., 2021). Agile allows the team to efficiently build CRUD functionality, regex filtering, scheduled reporting, and secure database operations while still flexible to future changes.

Compared to traditional models like Waterfall, Agile provides greater adaptability and stronger defect management (Sommerville, 2016). DevOps further improves deployment, monitoring, and system reliability after release. Although the process requires effective communication and organization, its advantages make it highly effective for developing and supporting a complex, large-scale research application.

AGILE + DEVOPS SOFTWARE LIFE CYCLE MODEL
RESEARCH DATA MANAGEMENT SYSTEM APPLICATION



References

Kim, G., Humble, J., Debois, P., & Willis, J. (2021). *The DevOps handbook* (2nd ed.). IT Revolution Press.

Schwaber, K. (2004). *Agile project management with Scrum*. Microsoft Press.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.